Numerical Analysis Assignment 2 — Report

Name	Student ID
Burhanuddin Murtaza	32400
Syed Mujtaba Zaidi	32436
Sarmad Ansari	32415
Hamza Faisal Ahmed	32406
Muhammad Hussam Zubair	32416

Overview

This assignment builds root-finding tools from scratch and uses them to study when and why they work. `fixed_point.py` contains a generic fixed-point solver and the order-of-convergence estimator `estimate_order()`. `newton.py` contains Newton's Method with diagnostics (analytic or central-difference derivative, zero-derivative guard, cycle detection). `multiplicity.py` contains `estimate_multiplicity()`, `modified_newton()` and the decision routine `auto_newton()` for multiple roots. `applications.py` applies all of them to Kepler's equation and to the van der Waals critical point of CO_2.

1 Problem 1 — Generic Fixed-Point Iteration

1.1 Method

The equation $f(x) = 0$ is rewritten as $x = g(x)$ and iterated as

$$p_{n+1} = g(p_n), \qquad n = 0, 1, 2, \ldots$$

By the Fixed-Point Theorem the iteration converges to $p = g(p)$ when $|g'(p)| < 1$ near p, and the asymptotic error constant is $|g'(p)|$. The empirical order of convergence is estimated from three consecutive errors:

$$\alpha \approx \frac{\ln(|e_{n+1}| \,/\, |e_n|)}{\ln(|e_n| \,/\, |e_{n-1}|)},$$

with $e_n = |p_n - p|$ when the true root is supplied and $e_n \approx |p_{n+1} - p_n|$ otherwise.

1.2 Results

Test function $f(x) = x^3 + 4x^2 - 10$, root $p \approx 1.365230013$, $p_0 = 1.5$, `tol` $= 10^{-8}$. Column α_{true} uses `true_value` $= 1.365230013$; column α_{diff} uses successive differences only (the value stored in `order_estimate` by `fixed_point_iteration`).

i	$g_i(x)$	$\lvert g_i'(p)\rvert$	Outcome	Iter.	α_{true}	α_{diff}
1	$x - x^3 - 4x^2 + 10$	15.513	Diverges ($\lvert p_5\rvert > 10^{10}$)	5	—	—
2	$\sqrt{10/x - 4x}$	3.430	Stops: domain error at step 3	2	—	—
3	$\frac{1}{2}\sqrt{10 - x^3}$	0.512	Converges	27	1.012	1.000
4	$\sqrt{10/(4+x)}$	0.127	Converges	10	0.9995	1.000
5	$x - \dfrac{x^3 + 4x^2 - 10}{3x^2 + 8x}$	0	Converges	4	1.969	2.000

Table 1: Fixed-point results for the five rearrangements.

The first iterates show the failures. For g_1: $1.5 \to -0.875 \to 6.73 \to -469.7 \to 1.03 \times 10^8 \to -1.08 \times 10^{24}$. For g_2: $1.5 \to 0.8165 \to 2.9969$, after which $10/x - 4x < 0$ and `math.sqrt` raises a domain error, which the solver catches and reports (`converged=False`, `root=None`).

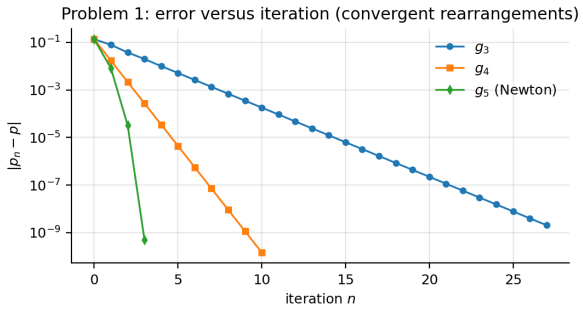


Figure 1: Error versus iteration for the convergent rearrangements. g_3 and g_4 are straight lines on the log scale (linear convergence, slopes $\ln 0.512$ and $\ln 0.127$); g_5 drops quadratically.

1.3 Ranking and discussion

Ranking (worst to best): g_1, g_2 (fail) $\rightarrow$ g_3 (slow) $\rightarrow$ g_4 (fast) $\rightarrow$ g_5 (fastest).

- g_1 **and** g_2**:** $|g_1'(p)| = 15.51$ and $|g_2'(p)| = 3.43$ are both greater than 1, so p is a repelling fixed point and the iterates move away from it. g_1 is stopped by the divergence test; g_2 leaves the domain of the square root.
- g_3**:** $|g_3'(p)| = 0.512 < 1$, so the error shrinks by about 0.512 per step (linear), needing 27 iterations.
- g_4**:** $|g_4'(p)| = 0.127 < 1$, so the error shrinks by about 0.127 per step (a factor of about 8), needing 10 iterations.
- g_5**:** $g_5'(p) = 0$, which gives quadratic convergence in only 4 iterations.

1.4 Connection between g_5 and Newton's Method

g_5 is Newton's Method for f. Since $f'(x) = 3x^2 + 8x$,

$$g_5(x) = x - \frac{f(x)}{f'(x)} = x - \frac{x^3 + 4x^2 - 10}{3x^2 + 8x}.$$

For any $g(x) = x - f(x)/f'(x)$,

$$g'(x) = 1 - \frac{f'(x)^2 - f(x)f''(x)}{f'(x)^2} = \frac{f(x)\,f''(x)}{[f'(x)]^2},$$

so $f(p) = 0$ and $f'(p) \neq 0$ give $g_5'(p) = 0$. A vanishing first derivative at the fixed point removes the linear term of the error recursion, $e_{n+1} \approx \frac{1}{2}g''(p)e_n^2$, which is exactly quadratic convergence.

1.5 Design decisions

- **Error floor in `estimate_order`.** With `true_value` given, errors below 10^{-7} are discarded. The supplied value 1.365230013 differs from the true root 1.3652300134141 by 4.1×10^{-10}, so below about 10^{-7} that error contaminates e_n by more than 0.4% and the log-ratios stop being reliable. This is why α_{true} is 1.012 and 0.9995 rather than exactly 1, and why the solver's own estimate α_{diff} is cleaner. Without `true_value` the floor is 10^{-12}, about four orders of magnitude above machine epsilon (2.2×10^{-16}), so the successive differences keep at least four reliable digits.
- **Guards in `estimate_order`.** It returns `order=None` with a note if there are fewer than 3 points, fewer than 3 usable errors, non-finite or non-decreasing errors, or if $|\ln(e_n/e_{n-1})| < 10^{-12}$ (division by $\ln 1 \approx 0$).
- **Successive differences.** For linear convergence $|p_{n+1} - p_n| \approx (1 - \lambda)\,|e_n|$, a constant multiple of the true error, so the ratio-of-logs gives the correct order asymptotically; early iterates are biased. Table 1 confirms this: $\alpha_{\text{diff}} = 1.000$ to three digits for g_3, g_4.
- **Divergence detection.** The solver stops if $|p_n| > 10^{10}$, if an iterate is `nan`/`inf`, or if evaluating g raises an exception; every iterate and difference is stored in `history` in all cases.

1.6 Limitations

- The oscillation / no-shrinkage detector suggested in the specification is *not* implemented; a sequence that oscillates without growing past 10^{10} runs until `max_iter`.
- The solver cannot keep an iteration inside the domain of g (e.g. g_2); it only catches the error and reports it.
- If $|g'(p)|$ is close to 1 convergence is very slow and the run can reach `max_iter`.
- `estimate_order` uses the last three errors of the initial strictly decreasing stretch, so very short runs (such as g_5) give a coarse estimate.

1.7 AI Usage Declaration — Problem 1

Tool used	Part of the assignment	Nature of the assistance	Est. % AI-assisted
ChatGPT, Gemini	Problem 1 code	Generating docstrings for both functions, help generating the `if __name__ == "__main__"` block, and code debugging (fixing syntax)	15%
ChatGPT	Problem 1 report	Reviewed the report section and suggested corrections (results table, explanation of g_1/g_2, error-floor discussion)	10%
Claude	Problem 1 report	Drafted the final report text and LaTeX formatting, generated Figure 1	30%

2 Problem 2 — Newton's Method with Convergence Diagnostics

2.1 Method

$$p_{n+1} = p_n - \frac{f(p_n)}{f'(p_n)}, \qquad f'(p_n) \approx \frac{f(p_n + h) - f(p_n - h)}{2h} \quad (h = 10^{-6}, \text{ numeric option}).$$

For a simple root, $|e_{n+1}| \approx C\,|e_n|^2$ with $C = |f''(p)|\,/(2\,|f'(p)|)$. `newton()` stores $(n, p, f_p, f'_p, \text{diff})$ at every step, raises `ZeroDivisionError` (reporting the iteration and the value of $f'(p_n)$) if $|f'(p_n)| < 10^{-12}$, stops with `reason="cycle detected"` on a persistent 2-cycle, and imports `estimate_order` from `fixed_point.py` (successive-difference mode) rather than copying it.

2.2 Test (i): $f(x) = x^2 - 2$, $p_0 = 1$

Derivative	Iterations	Root	$\lvert p - \sqrt{2}\rvert$	Est. order
Analytic, $f' = 2x$	5	1.4142135623730951	0 (to double precision)	2.00001
Numeric (central diff.)	5	1.4142135623730951	0 (to double precision)	2.00003

Table 2: Analytic versus numeric derivative for $x^2 - 2$.

The error sequence 4.1×10^{-1}, 8.6×10^{-2}, 2.5×10^{-3}, 2.1×10^{-6}, 1.6×10^{-12}, 0 squares at every step ($e_2 \approx 0.354\,e_1^2$ with $C = 1/(2\sqrt{2}) = 0.354$), confirming quadratic convergence. The two derivative options need the *same number of iterations* (difference 0) and reach the *same final accuracy* (identical doubles). The only visible difference is in the early iterates, e.g. $p_1 = 1.4999999999856$ versus 1.5, a gap of 1.4×10^{-11}. For a quadratic the central difference has zero truncation error ($f''' = 0$), so this gap is pure round-off of size $\approx \varepsilon\,|f|\,/h$; Newton's quadratic contraction erases it within one step.

2.3 Test (ii): $f(x) = \cos x - x$, $p_0 = 0.5$

Derivative	Iterations	Root	Est. order
Analytic, $f' = -\sin x - 1$	5	0.7390851332151607	1.9982
Numeric (central diff.)	5	0.7390851332151607	1.9982

Table 3: Newton's Method on $\cos x - x$.

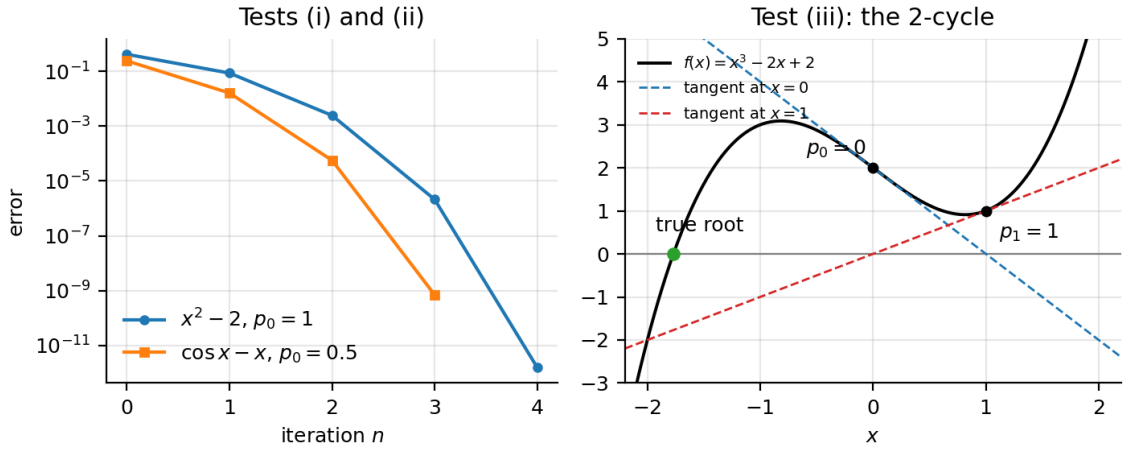


Figure 2: Left: quadratic convergence in tests (i) and (ii). Right: the tangent lines of test (iii) that produce the cycle $0 \leftrightarrow 1$.

Both runs converge in 5 iterations to 0.7390851332 with empirical order ≈ 2, as expected for a simple root ($f'(p) = -1.674 \neq 0$).

2.4 Test (iii): the pathological case $f(x) = x^3 - 2x + 2$, $p_0 = 0$

Hand verification. Here $f'(x) = 3x^2 - 2$. At $p_0 = 0$: $f(0) = 2$, $f'(0) = -2$, so

$$p_1 = 0 - \frac{2}{-2} = 1.$$

At $p_1 = 1$: $f(1) = 1 - 2 + 2 = 1$, $f'(1) = 3 - 2 = 1$, so

$$p_2 = 1 - \frac{1}{1} = 0 = p_0.$$

Hence the iteration alternates $0, 1, 0, 1, \ldots$ forever and never approaches the real root $p \approx -1.76929$.

Solver output. With the analytic derivative the solver stops after 12 iterations with `converged=False`, `reason="cycle detected"`, `root=None`; with the numeric derivative it stops after 13 (the iterates alternate between 1.0000000000268 and 2.5×10^{-10} because the central difference carries round-off). It neither hangs nor returns a wrong "root".

Geometric explanation. Newton's Method replaces f by its tangent at p_n, $y = f(p_n) + f'(p_n)(x - p_n)$, and takes its x-intercept as p_{n+1}. The tangent at $x = 0$ is $y = 2 - 2x$, which meets the axis at $x = 1$; the tangent at $x = 1$ is $y = x$, which meets the axis at $x = 0$ (Figure 2, right). The curve has a local minimum at $x = \sqrt{2/3} \approx 0.816$ with value $0.911 > 0$, so on $[0, 1]$ the graph never reaches the axis and each tangent overshoots across the valley instead of heading toward the only real root at $x \approx -1.769$.

A starting value that converges. From $p_0 = 2$ the iterates are

$$2 \to 1.4 \to 0.899 \to -1.289 \to -2.106 \to -1.829 \to -1.772 \to -1.7693 \to -1.76929,$$

converging in 9 iterations to -1.7692923542. From $p_0 = -2$ (already on the correct side of the local maximum) it converges in 5 iterations ($-2 \to -1.8 \to -1.76995 \to -1.76929$) with estimated order 2.003.

2.5 Design decisions

- **Derivative threshold:** `MIN` $= 10^{-12}$. Below this the Newton step is dominated by round-off in f', so a `ZeroDivisionError` naming the iteration and $f'(p_n)$ is raised (e.g. $f = x^2 + 1$, $p_0 = 0$: "at iteration: 1, the value of f'(p_n) becomes 0.0").
- **Cycle-detection tolerance:** $|p_{n+2} - p_n| < 10^{-12}$ (the same constant), far above round-off but far below any step a progressing iteration takes.
- **Consecutive matches:** 11 consecutive pairs (> 10) are required. A convergent run reaches $|p_{n+1} - p_n| <$ `tol` well before 11 near-equal pairs accumulate, so there are no false positives for `tol` $\geq 10^{-12}$, yet a true cycle is stopped after about a dozen iterations instead of `max_iter`.
- **Order estimate:** from successive differences, via the imported `estimate_order`.

2.6 Limitations

- When a run does not converge, `root` is `None` (the last iterate is only available in `history`).
- In each `history` entry, `f_p` and `fprime_p` belong to the *previous* iterate, not to the iterate stored as `p` in that entry.
- Only 2-cycles are detected; longer cycles or slow chaotic wandering run to `max_iter`.
- The numeric derivative is sensitive to h: round-off $\sim \varepsilon/h$ dominates for very small h, truncation $\sim h^2$ for large h.

2.7 AI Usage Declaration — Problem 2

Tool used	Part of the assignment	Nature of the assistance	Est. % AI-assisted
Claude	Problem 2 code	Helped format the verbose mode and the output strings	10%
Claude	Problem 2 code	Reviewed the final version for errors	5%
Claude	Problem 2 report	Drafted the report text and LaTeX formatting, generated Figure 2	15%

3 Problem 3 — Multiplicity Detection and the Modified Newton's Method

3.1 Method

If p is a root of multiplicity $m > 1$, Newton's Method converges only linearly with $\lambda = (m-1)/m$. The ratio $r_n = (p_{n+1} - p_n)/(p_n - p_{n-1}) \to \lambda$, so $m \approx 1/(1 - r_n)$. Two corrections restore quadratic convergence:

$$p_{n+1} = p_n - m\frac{f(p_n)}{f'(p_n)} \quad \text{(known } m\text{)}, \qquad p_{n+1} = p_n - \frac{f(p_n)f'(p_n)}{[f'(p_n)]^2 - f(p_n)f''(p_n)} \quad (u = f/f').$$

`auto_newton()` runs 6 Newton steps, estimates the order from them, and switches to the $u = f/f'$ form from the current iterate if the order is below 1.5.

3.2 Required test: $f(x) = (x-2)^3(x+1)$

(a) Ordinary Newton from $p_0 = 2.5$. After the maximum of 30 iterations (with `tol` $= 10^{-8}$) the iterate is 2.0000028 and the run has *not* converged. The empirical order is 1.0000. The ratio r_n starts at 0.691 and settles at 0.666667, the theoretical $\lambda = 2/3$ (sequence $0.6908, 0.6844, 0.6793, 0.6755, \ldots, 0.6672, 0.6670$). Ordinary Newton is also unusable at tighter tolerances: it needs 20 iterations for 10^{-4}, 31 for 10^{-6}, and for `tol` $\le 10^{-7}$ the derivative guard fires first (`ZeroDivisionError` at iteration 37, $f' = 5.5\times10^{-13}$), because $f'(x) = (x-2)^2(4x+1)$ itself vanishes at the multiple root.

(b) `estimate_multiplicity` on the same data:

m_estimate	m_rounded	ratio_used
3.0	3	0.6667

(`diagnostic_note`: "Stable linear convergence observed with ratio $r_n = 0.6667$; $\lambda = (m-1)/m$ yields $m \approx 3.0000 \to$ rounded $m = 3$.")

(c) Modified Newton from $p_0 = 2.5$ (`tol` $= 10^{-10}$):

Method	Iterations	Converged	Est. order	Final error
Ordinary Newton	> 30 (cap)	No	1.000	2.8×10^{-6}
Modified, $m = 3$ known	4	Yes	1.966	3.6×10^{-10}
Modified, $u = f/f'$	4	Yes	1.872	2.4×10^{-10}

Table 4: Ordinary versus modified Newton on the triple root.

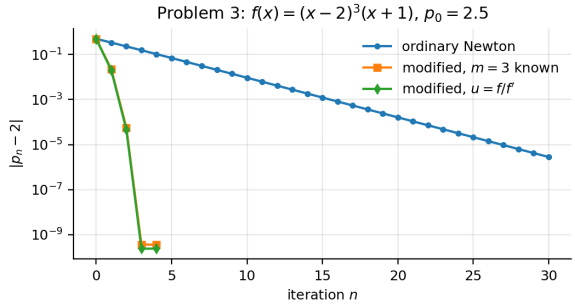


Figure 3: Ordinary Newton (linear, slope $\ln \frac{2}{3}$) versus the two modified methods (quadratic).

The errors of the known-m run are 5.0×10^{-1}, 2.3×10^{-2}, 5.7×10^{-5}, 3.6×10^{-10}; the ratios $e_{n+1}/e_n^2 = 0.091, 0.110, 0.111$ are constant, so convergence is quadratic. The $u = f/f'$ run is the same (0.081, 0.113, 0.111). Both therefore replace roughly 40 linear steps (ordinary Newton did not finish in 30, and for tight tolerances its derivative guard stops it at iteration 37) by 4 quadratic steps, and the second needs no knowledge of m.

(d) `auto_newton` without being told the root or m:

Start	**Root found**	`method_used`	**Iter.**	`multiplicity_info`
$p_0 = 2.5$	1.9999999937	`modified_newton`	9	$m_{est} = 3.1184$, $m = 3$, $r = 0.6793$
$p_0 = -0.5$	-1.0	`newton`	7	`None`

Table 5: Automatic method selection.

For the simple root ($p_0 = -0.5$) the probe shows quadratic behaviour (final order 2.0004), so ordinary Newton is kept; for the triple root the probe order is below 1.5, the multiplicity is recovered as $m = 3$, and the solver switches to $u = f/f'$. The reported order 1.010 for the first run is computed over the *whole* history, which includes the six linear probe steps, so it understates the quadratic phase.

3.3 Edge cases

- $m = 1$**:** `modified_newton` with $m = 1$ on $x^2 - 2$ versus ordinary `newton`: maximum difference between iterates = 0.0 (identical).
- $u'(p_n) \approx 0$**:** for $f = e^x$, $u'(x) = 1 - e^x e^x/(e^x)^2 = 0$ identically; the solver raises `ZeroDivisionError`: "at iteration: 1, $u'(p_n) = 0.000 \times 10^0$ falls below threshold 10^{-12}. Modified Newton step is undefined." No `nan` is produced.
- **Noise near machine epsilon:** the order estimate reuses the floors of `estimate_order`; `modified_newton` also stops when $|f(p)| < 10^{-15}$.
- **High multiplicity.** For $f = (x-2)^m(x+1)$, $p_0 = 2.5$, we ran `estimate_multiplicity` on Newton histories of different probe lengths:

m	6 iterations	10 iterations	15 iterations	25 iterations
3	3.118	3.026	3.004	3.000
4	4.150	4.052	4.013	4.001
5	5.171	5.077	5.026	5.003
6	6.185	6.097	6.041	6.007

Table 6: Unrounded `m_estimate` versus probe length (true m in the first column).

With 6 probe iterations the estimate is biased high by about 0.15–0.19 but still rounds to the correct integer for $m = 3, \ldots, 6$. The bias grows slowly with m (0.12 at $m = 3$ to 0.19 at $m = 6$) because $dm/dr = 1/(1-r)^2 = m^2$ amplifies the residual error in r_n. Extrapolating this trend, rounding would eventually fail for larger m (we did not test this). The remedy would be to lengthen the probe (the table shows the bias roughly halves every 4–5 extra iterations), to start probing closer to the root, or to accelerate the ratios (e.g. with Aitken's Δ^2).

3.4 Design decisions

- `probe_iters` $= 6$**:** the experiment above shows 6 steps already give the correct rounded multiplicity for $m \leq 6$, while keeping the probe cheap. It is also the point where the order estimate separates linear (≈ 1) from quadratic (≈ 2) behaviour.
- `order_threshold` $= 1.5$**:** midway between linear (1) and quadratic (2).
- u' **guard:** the same 10^{-12} threshold as in Problem 2, so that a vanishing u' raises an informative error instead of a silent `nan`.

3.5 Limitations

- Stopping at $|f(p)| < 10^{-15}$ on a multiple root can return a less accurate iterate than `tol` suggests (final errors 3.6×10^{-10} and 2.4×10^{-10} in (c), and 6.3×10^{-9} for `auto_newton`), because $f \sim e^3$ is tiny even when e is not.
- The ordinary-Newton derivative guard fires before convergence on a multiple root when `tol` $\lesssim 10^{-7}$.
- The order reported by `auto_newton` includes the linear probe phase, and the multiplicity estimate degrades for large m or noisy ratios.

3.6 AI Usage Declaration — Problem 3

Tool used	**Part of the assignment**	**Nature of the assistance**	**Est. % AI-assisted**
Claude	Problem 3 code	Ai was used to generate doc strings. Testing code, debugging syntax	15%
Claude	Problem 3 report	Drafted the report text and LaTeX formatting, generated Figure 3	15%

4 Problem 4 — Two Real-World Applications

4.1 Application A: Kepler's Equation

$M = E - e\sin E$ with $M = 1.0$ rad, $E_0 = M$, `tol` $= 10^{-10}$. Fixed point: $g(E) = M + e\sin E$, $g'(E) = e\cos E$. Newton: $f(E) = E - e\sin E - M$, $f'(E) = 1 - e\cos E$.

		Fixed point		**Newton**		
Body	e	Iter.	Order	Iter.	Order	E^*
Mars	0.0934	8	1.000	4	1.994	1.0824839537
Mercury	0.2056	10	1.000	4	1.984	1.1909447672
Halley's Comet	0.967	21	1.000	6	2.000	1.9114369765

Table 7: Kepler's equation, $M = 1$ rad.

Body	e	$\lvert g'(E^*)\rvert = e\,\lvert\cos E^*\rvert$	Observed ratio of successive differences
Mars	0.0934	0.0438	0.044
Mercury	0.2056	0.0762	0.076
Halley's Comet	0.967	0.3231	0.32

Table 8: The fixed-point error ratio equals $|g'(E^*)|$.

Why the fixed-point rate depends on e. Since $|g'(E)| = e\,|\cos E| \leq e$, the error contracts by a factor $|g'(E^*)| \leq e$ per step, as Table 8 confirms (observed ratio $= |g'(E^*)|$). For the nearly circular orbit of Mars the factor is 0.044 (8 iterations); for Mercury 0.076 (10); for Halley's Comet it is 0.32 (21).

Why Halley's Comet is much worse for fixed point. The contraction factor tends to e as $E^* \to 0$ or π, i.e. near periapsis (M small) or apoapsis, and e itself is close to 1 for Halley's Comet. With $M = 1$ rad the comet happens to have $\cos E^* = -0.33$, so the penalty is moderate (21 versus 6 iterations). As an extra experiment (not required) with $e = 0.967$: $M = 0.5$ needs 13 fixed-point versus 7 Newton iterations; $M = 0.1$ needs 62 versus

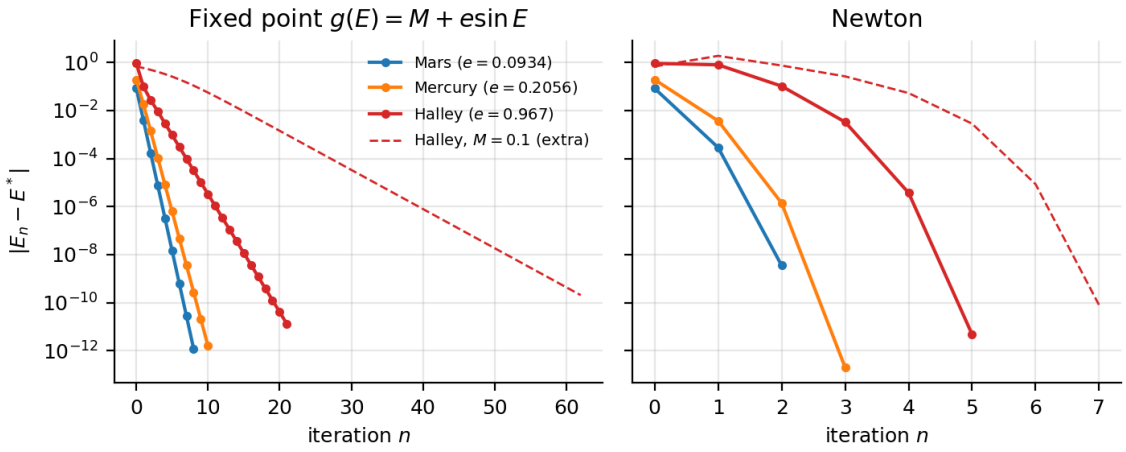


Figure 4: Error versus iteration. Fixed point (left) is linear with slope $\ln|g'(E^*)|$; Newton (right) is quadratic. Dashed: an extra experiment, Halley's Comet at $M = 0.1$.

8 ($|g'(E^*)| = 0.687$); and for $M = 0.01$ fixed point has not converged after 100 iterations ($|g'(E^*)| = 0.940$) while Newton needs 6. Newton's cost grows only mildly (4, 4 and 6 iterations at $M = 1$ rad; 8 at $M = 0.1$) because the denominator $1 - e\cos E \geq 1 - e > 0$ and the iteration stays quadratic.

Recommendation. For a propagator that must handle all eccentricities I would deploy Newton's Method: 4–6 iterations in every case here, quadratic convergence, and a rate that does not degrade as $e \to 1$. Fixed point is cheaper per step but its iteration count grows without bound as $e \to 1$ and $M \to 0$. For e near 1 a safeguarded starting value would be prudent, but we did not test one.

4.2 Application B: Critical point of a real gas (van der Waals)

For CO_2: $a = 3.592\ \text{atm}\,\text{L}^2/\text{mol}^2$, $b = 0.04267\ \text{L/mol}$, $R = 0.082057\ \text{L}\,\text{atm}/(\text{mol}\,\text{K})$. The cubic in V is $PV^3 - (Pb + RT)V^2 + aV - ab = 0$ and

$$V_c = 3b, \qquad P_c = \frac{a}{27b^2}, \qquad T_c = \frac{8a}{27Rb}.$$

(a) Critical constants.

Quantity	Value	Units
T_c	303.965485	K
P_c	73.068010	atm
V_c	0.128010	L/mol

(b) Triple-root check. With $f(V) = P_cV^3 - (P_cb + RT_c)V^2 + aV - ab$,

$$f'(V) = 3P_cV^2 - 2(P_cb + RT_c)V + a, \qquad f''(V) = 6P_cV - 2(P_cb + RT_c).$$

Substituting $P_c = a/(27b^2)$ and $RT_c = 8a/(27b)$ gives $P_cb + RT_c = a/(3b)$ and the exact factorisation

$$f(V) = \frac{a}{27b^2}(V - 3b)^3 = P_c\,(V - V_c)^3,$$

so V_c is a root of multiplicity exactly 3, with $f'''(V_c) = 6P_c = 438.408 \neq 0$. Numerically:

Quantity	Value at V_c	Scale of the terms	Relative size
$f(V_c)$	0	4.60×10^{-1}	0
$f'(V_c)$	4.4×10^{-16}	7.18	6.2×10^{-17}
$f''(V_c)$	7.1×10^{-15}	56.1	1.3×10^{-16}

Table 9: All three vanish to within machine precision.

(c) Ordinary Newton from $V_0 = 1.2V_c = 0.153612$. The error ratio is $e_{n+1}/e_n = 0.6667$ for the first twenty iterations (it is exactly 2/3 because $f/f' = (V - V_c)/3$), and the order estimated on those iterates is 1.0002, i.e.

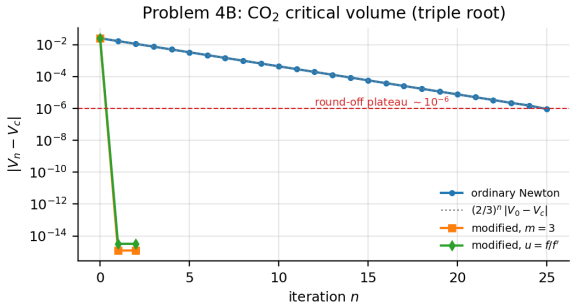


Figure 5: Ordinary Newton follows the line $(2/3)^n\,|V_0 - V_c|$ until round-off stops it near 10^{-6}; the modified methods reach machine precision in one step.

linear, consistent with $m = 3$. Note that `estimate_order` applied to the full 25-iterate history prints 0.198 instead: below $|V - V_c| \approx 10^{-6}$ the evaluation of f is dominated by round-off (the plateau $\sim (\varepsilon \cdot \text{scale}/P_c)^{1/3} \approx 10^{-6}$ in Figure 5), so the tail of the history is noise. The run with `max_iter` = 100 "converges" after 27 iterations only because a computed step becomes exactly 0; its root is accurate to just 6.2×10^{-7}, not 10^{-10}.

(d) Multiplicity estimate and modified Newton.

Method	Iterations	$\lvert V - V_c \rvert$	Notes
`estimate_multiplicity`	—	—	$m_{est} = 3.0011$, $m = 3$, $r = 0.6668$
Ordinary Newton	25 (cap) / 27	6×10^{-7}	order ≈ 1
`modified_newton`, $m = 3$	2	1.2×10^{-15}	order not estimable
`modified_newton`, $u = f/f'$	2	3.2×10^{-15}	order not estimable
`auto_newton`	8	4.5×10^{-13}	switched to modified Newton, $m = 3$

Table 10: Triple root of the van der Waals cubic.

Because $f(V) = P_c(V - V_c)^3$ exactly, the correction $V - 3f/f'$ lands on V_c in a *single* step (error $2.6 \times 10^{-2} \to 10^{-15}$); the second iteration only confirms $|f| < 10^{-15}$. Convergence is therefore not merely quadratic but finite-step, and with fewer than three iterates `estimate_order` correctly returns `None`. The `auto_newton` order (1.000) again covers the linear probe phase. Quadratic convergence is thus restored: 2 iterations (and 10^{-15} accuracy) versus 27 iterations (and 6×10^{-7} accuracy).

4.3 Design decisions

- The triple-root check uses a *relative* measure (value divided by the largest term in the polynomial), because absolute values of f' and f'' near 10^{-15} are only meaningful against the size of the terms that cancel.
- For part (c) a 25-iteration cap is used for the order estimate so the run stops near the round-off plateau; the order is additionally computed on the first 20 iterates, where the data are clean.
- f'' is supplied analytically ($6P_cV - 2(P_cb + RT_c)$); Kepler's equation uses the analytic $f' = 1 - e\cos E$.

4.4 Limitations

- A triple root of a double-precision polynomial cannot be located better than about $\varepsilon^{1/3} \approx 10^{-6}$ by plain Newton; only the modified methods avoid this.
- Fixed-point iteration for Kepler's equation fails to converge in 100 iterations when e is close to 1 and M is small (e.g. $e = 0.967$, $M = 0.01$).
- The order reported by `auto_newton` includes the probe phase, and `estimate_order` gives no value when a method terminates in fewer than three iterations.

4.5 AI Usage Declaration — Problem 4

Tool used	Part of the assignment	Nature of the assistance	Est. % AI-assisted
Claude	Problem 4 code	doc strings and print statements with the fstrings	20%
Claude	Problem 4 report	Drafted the report text and LaTeX formatting, generated Figures 4 and 5	25%

5 Overall Design Decisions and Limitations

Overall design decisions (details in Sections 1.5, 2.5 and 3.4): error floor in `estimate_order` 10^{-7} with `true_value` and 10^{-12} without; divergence threshold $|p_n| > 10^{10}$; derivative threshold 10^{-12}; cycle tolerance 10^{-12} with 11 consecutive matches; `probe_iters` $= 6$; `order_threshold` $= 1.5$.

Overall limitations. (i) No oscillation detector in the fixed-point solver. (ii) Only 2-cycles are detected in Newton's Method. (iii) For multiple roots the $|f| < 10^{-15}$ shortcut and the 10^{-12} derivative guard can stop a run with accuracy different from `tol`. (iv) The `auto_newton` order includes the probe phase. (v) Multiplicity estimation depends on a clean linear phase and becomes biased for large m.

6 Summary of AI Usage

AI use is declared per problem in Sections 1.7, 2.7, 3.6 and 4.5 (separate tables for the code and for the report text of each problem). Outside those tables no AI assistance was used.